

Top 20 React Interview Questions: Detailed Answers with Practical Code

Experienced developers • Senior level

React 18/19 concepts

This guide expands the 20 questions from our previous discussion. Each answer includes a technical explanation, practical React code, production considerations, and an interview-ready summary.

The examples use modern React functional components, JavaScript, and JSX. Where relevant, I'll also cover enterprise application scenarios such as dashboards, authentication, API integration, and performance optimization.

Interview tip: For senior positions, explain not only how a feature works but also when to use it, its limitations, and how you would troubleshoot it in production.

1. How does React work, and what is the Virtual DOM?

Core React concepts

Answer

React is a declarative JavaScript library for building user interfaces using reusable components.

When a component's state changes, React performs three conceptual steps:

- 1. Trigger:** A state update schedules rendering.
- 2. Render:** React calls components to calculate the next UI.
- 3. Commit:** React applies necessary changes to the actual DOM.

The Virtual DOM is an informal term for React's in-memory representation of UI. React compares the previous and next component trees through reconciliation.

Practical example

JavaScript

```
import { useState } from "react";

function Counter() {
  const [count, setCount] = useState(0);

  return (
```

```

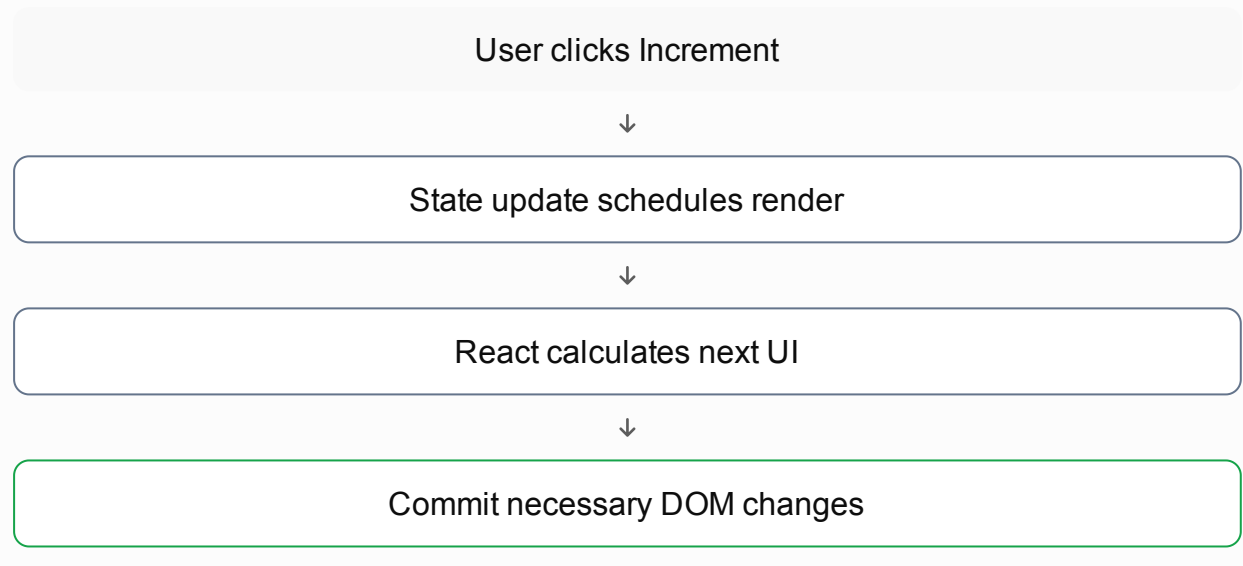
<div>
  <h2>Count: {count}</h2>
  <button onClick={ () => setCount(c => c + 1)}>
    Increment
  </button>
</div>
);
}

export default Counter;

```

When you click Increment, React renders the component with the updated count and commits the necessary DOM changes. It does not need to recreate the entire page.

React update lifecycle



Senior-level point: React rendering does not necessarily mean a browser DOM mutation. A component can render without any actual DOM changes.

Interview-ready answer: React uses a declarative component architecture and reconciliation to calculate UI updates. It separates rendering from committing changes, allowing React to avoid unnecessary DOM operations.

2. What is the difference between useState and useReducer?

State management

Answer

Both hooks manage component state, but they are suited to different levels of complexity.

useState

useReducer

Simple state updates

Complex transitions

Direct setter

Dispatch actions

Less boilerplate

Centralized update logic

Forms, counters, toggles

Workflows, carts, complex forms

Example 1: `useState`

JavaScript

```
import { useState } from "react";

function Counter() {
  const [count, setCount] = useState(0);

  return (
    <button onClick={() => setCount(c => c + 1)}>
      Count: {count}
    </button>
  );
}
```

Example 2: `useReducer`

JavaScript

```
import { useReducer } from "react";

const initialState = { count: 0 };

function reducer(state, action) {
  switch (action.type) {
    case "increment":
      return { ...state, count: state.count + 1 };

    case "decrement":
      return { ...state, count: state.count - 1 };

    case "reset":
      return initialState;

    default:
      return state;
  }
}

function Counter() {
  const [state, dispatch] = useReducer(
```

```

    reducer,
    initialState
  );

  return (
    <>
      <h2>{state.count}</h2>

      <button onClick={ () =>
        dispatch({ type: "increment" })
      }>
        +
      </button>

      <button onClick={ () =>
        dispatch({ type: "decrement" })
      }>
        -
      </button>

      <button onClick={ () =>
        dispatch({ type: "reset" })
      }>
        Reset
      </button>
    </>
  );
}

```

Real-world scenario: An insurance claim form might have actions such as `ADD_DOCUMENT`, `UPDATE_CLAIM`, `VALIDATE_CLAIM`, and `RESET`. A reducer keeps these related transitions together.

Reducers should be pure functions and must return new state objects rather than mutating existing state.

Interview-ready answer: I use `useState` for simple independent values and `useReducer` when state transitions become complex or multiple actions affect related fields.

3. Explain `useEffect` and its dependency array.

Hooks and lifecycle

Answer

`useEffect` synchronizes a component with an external system, such as a browser event, network connection, or subscription.

The dependency array determines when the effect needs to run again.

Syntax

Behavior

```
useEffect(fn)
```

After every committed render

```
useEffect(fn, [])
```

On mount; cleanup on unmount

```
useEffect(fn, [id])
```

On mount and when `id` changes

In development Strict Mode, React may perform an additional setup-and-cleanup cycle to detect effect bugs.

Practical example: Fetching customer details

JavaScript

```
import { useEffect, useState } from "react";

function CustomerDetails({ customerId }) {
  const [customer, setCustomer] = useState(null);

  useEffect(() => {
    const controller = new AbortController();

    setCustomer(null);

    async function loadCustomer() {
      try {
        const response = await fetch(
          `/api/customers/${customerId}`,
          { signal: controller.signal }
        );

        if (!response.ok) {
          throw new Error("Request failed");
        }

        const data = await response.json();
        setCustomer(data);
      } catch (error) {
        if (error.name !== "AbortError") {
          console.error(error);
        }
      }
    }

    loadCustomer();

    return () => controller.abort();
  }, [customerId]);
```

```
return <div>{customer?.name ?? "Loading..."}</div>;  
}
```

request prevents its response from updating the component after a newer request has started.

Production applications should also manage error and loading states explicitly.

Interview-ready answer: `useEffect` handles synchronization with external systems. I declare all reactive dependencies and implement cleanup for subscriptions, timers, and requests. I avoid using effects for values that can be calculated during rendering.

4. What is the difference between `useMemo` and `useCallback`?

Performance optimization

Answer

Both hooks preserve values between renders when their dependencies remain unchanged.

Hook	Caches
<code>useMemo</code>	Result of a calculation
<code>useCallback</code>	Function reference

Example: Filtering a large product list

JavaScript

```
import { useMemo, useCallback, useState } from "react";  
  
function ProductList({ products }) {  
  const [search, setSearch] = useState("");  
  
  const filteredProducts = useMemo(() => {  
    return products.filter(product =>  
      product.name  
        .toLowerCase()  
        .includes(search.toLowerCase())  
    );  
  }, [products, search]);  
  
  const handleSelect = useCallback((id) => {  
    console.log("Selected product:", id);  
  }, []);  
}
```

```

return (
  <>
    <input
      value={search}
      onChange={e => setSearch(e.target.value)}
    />

    {filteredProducts.map(product => (
      <button
        key={product.id}
        onClick={() => handleSelect(product.id)}
      >
        {product.name}
      </button>
    ))}
  </>
);
}

```

Here, `useMemo` avoids repeating the filtering calculation when its dependencies are unchanged.

`useCallback` preserves the identity of `handleSelect`. This is particularly useful when passing it to memoized child components.

Senior-level consideration: Do not memoize everything. Memoization has costs, and the React Compiler may automate some optimizations in applications configured to use it. Profile before adding manual optimizations.

Interview-ready answer: `useMemo` caches calculation results, while `useCallback` caches function references. I use them when profiling or component memoization reveals a measurable benefit.

5. How do you optimize performance in a large React application?

Senior-level scenario

Answer

Performance optimization should begin with measurement rather than assumptions.

A practical strategy includes reducing unnecessary renders, splitting JavaScript bundles, virtualizing large lists, optimizing network requests, and measuring results.

Example: Rendering 10,000 records

Rendering thousands of DOM elements simultaneously can make scrolling and interactions slow.

A virtualization library renders only the visible portion of the list.

Install:

Bash

```
npm install react-window@1
```

Example using the react-window v1 API:

JavaScript

```
import { FixedSizeList } from "react-window";

function EmployeeList({ employees }) {
  return (
    <FixedSizeList
      height={500}
      width={400}
      itemCount={employees.length}
      itemSize={50}
      itemData={employees}
    >
      {({ index, style, data }) => (
        <div style={style}>
          {data[index].name}
        </div>
      )}
    </FixedSizeList>
  );
}
```

Note: This example targets react-window v1; newer major versions have a different API.

Instead of rendering all 10,000 employees, virtualization mounts only the visible rows plus an overscan buffer.

Other production techniques

- Use React DevTools Profiler to identify expensive renders.
- Split routes using `React.lazy` and `Suspense`.
- Debounce expensive searches when appropriate.
- Paginate or virtualize large datasets.
- Cache server responses and eliminate duplicate requests.
- Move expensive non-UI computation to Web Workers when justified.

Interview-ready answer: I start with React Profiler and browser performance measurements, identify the bottleneck, apply targeted optimizations such as virtualization or code splitting, and measure again to verify the improvement.

6. What causes unnecessary re-renders in React?

Rendering behavior

Answer

React components can render because their state changes, their parent renders, or a context value they consume changes.

Re-rendering is not automatically a performance problem. It becomes important when rendering work is expensive.

Example: Parent state triggers child rendering

JavaScript

```
import { useState } from "react";

function Parent() {
  const [count, setCount] = useState(0);

  return (
    <>
      <button onClick={() => setCount(c => c + 1)}>
        Count: {count}
      </button>

      <ExpensiveChild />
    </>
  );
}

function ExpensiveChild() {
  console.log("Child rendered");

  return <h2>Expensive component</h2>;
}
```

When the parent renders, React normally also renders `ExpensiveChild`, even though it receives no changing props.

Optimization using `React.memo`

JavaScript

```
import { memo } from "react";

const ExpensiveChild = memo(function ExpensiveChild() {
  console.log("Child rendered");

  return <h2>Expensive component</h2>;
});
```

Important limitation: `React.memo` does not prevent renders caused by the component's own state changes or context changes that it consumes.

Interview-ready answer: I identify re-renders using React DevTools Profiler, keep state close to its consumers, split large components, and use `React.memo` selectively when stable props make it beneficial.

7. Explain `React.memo`, `useMemo` and `useCallback` together.

Advanced optimization

Answer

These three APIs solve related but different performance problems.

API	Purpose
<code>React.memo</code>	Skip component rendering when props are unchanged
<code>useMemo</code>	Cache an expensive calculation
<code>useCallback</code>	Preserve a function reference

Practical example: Employee dashboard

JavaScript

```
import {
  memo,
  useMemo,
  useCallback,
  useState
} from "react";

const EmployeeTable = memo(function EmployeeTable({
  employees,
  onSelect
}) {
  console.log("Table rendered");
```

```

    return employees.map(employee => (
      <button
        key={employee.id}
        onClick={() => onSelect(employee.id)}
      >
        {employee.name}
      </button>
    ));
  });

function Dashboard({ employees }) {
  const [count, setCount] = useState(0);

  const activeEmployees = useMemo(() => {
    return employees.filter(e => e.active);
  }, [employees]);

  const handleSelect = useCallback((id) => {
    console.log("Employee:", id);
  }, []);

  return (
    <>
      <button onClick={() => setCount(c => c + 1)}>
        Counter: {count}
      </button>

      <EmployeeTable
        employees={activeEmployees}
        onSelect={handleSelect}
      />
    </>
  );
}

```

When the counter changes, the dashboard renders again. If the `employees` prop is stable, `useMemo` returns the same array, `useCallback` returns the same function, and `React.memo` can skip rendering the table.

Without stable props, memoizing the child might not help.

Interview-ready answer: `React.memo` memoizes a component's rendering behavior, `useMemo` caches computed values, and `useCallback` stabilizes function references. Together, they can reduce unnecessary work when component props remain stable.

8. What is the difference between Context API and Redux?

Enterprise state management

Answer

React Context provides a mechanism to pass values through a component tree without manually passing props at every level.

Redux is a state-management library that centralizes state updates through actions and reducers.

Context API	Redux
Built into React	External library
Provides values to descendants	Centralized store
Common for theme, locale, auth context	Useful for complex shared application state
Consumers update when context changes	Selectors allow targeted subscriptions

Example: Context API

JavaScript

```
import {
  createContext,
  useContext,
  useState
} from "react";

const ThemeContext = createContext(null);

function ThemeProvider({ children }) {
  const [theme, setTheme] = useState("light");

  return (
    <ThemeContext.Provider value={{ theme, setTheme }}>
      {children}
    </ThemeContext.Provider>
  );
}

function Header() {
  const { theme, setTheme } = useContext(ThemeContext);

  return (
    <button onClick={() =>
      setTheme(t => t === "light" ? "dark" : "light")}
    >
```

```
      Current theme: {theme}
    </button>
  );
}
```

Example: Redux Toolkit

JavaScript

```
import { createSlice, configureStore } from "@reduxjs/toolkit";

const cartSlice = createSlice({
  name: "cart",
  initialState: { items: [] },
  reducers: {
    addItem(state, action) {
      state.items.push(action.payload);
    }
  }
});

export const { addItem } = cartSlice.actions;

export const store = configureStore({
  reducer: {
    cart: cartSlice.reducer
  }
});
```

Redux Toolkit uses Immer internally, allowing this mutation-like reducer syntax while producing immutable state updates.

In components, connect the store using React Redux's `Provider`, `useSelector`, and `useDispatch`.

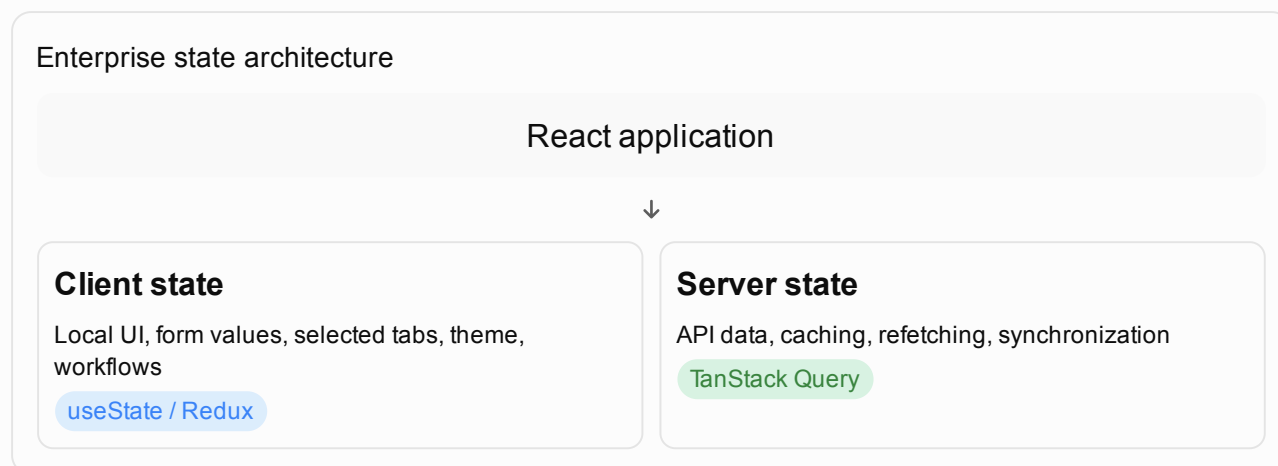
Interview-ready answer: Context is a dependency-sharing mechanism, while Redux provides a structured state-management architecture. I choose based on the complexity of shared state, update frequency, debugging requirements, and existing application conventions.

9. How do you manage global state in an enterprise React application?

Architecture scenario

Answer

The key architectural decision is to separate different categories of state rather than putting everything in one global store.



Example: Server-state management

JavaScript

```
import { useQuery } from "@tanstack/react-query";

function Customers() {
  const { data, isPending, error } = useQuery({
    queryKey: ["customers"],
    queryFn: async () => {
      const response = await fetch("/api/customers");

      if (!response.ok) {
        throw new Error("Failed to load customers");
      }

      return response.json();
    },
    staleTime: 60_000
  });

  if (isPending) return <p>Loading...</p>;
  if (error) return <p>Error: {error.message}</p>;

  return data.map(customer => (
    <p key={customer.id}>{customer.name}</p>
  ));
}
```

This example assumes the application is wrapped in `QueryClientProvider` with a configured `QueryClient`.

TanStack Query handles request caching, stale data, background refetching, and request deduplication for matching queries.

Enterprise decision: Keep component-only values local, use Context for broadly shared configuration, use Redux when complex client state benefits from centralized transitions, and use a server-state solution for API data.

Interview-ready answer: I classify state into local UI state, shared client state, and server state. I select the simplest appropriate tool for each category rather than storing all data globally.

10. Explain React reconciliation and the importance of keys.

React internals

Answer

Reconciliation is React's process for determining how the next UI tree relates to the previous one.

Component type and key help React determine whether to preserve component state or create a new component instance.

Incorrect example

JavaScript

```
{employees.map((employee, index) => (  
  <EmployeeRow  
    key={index}  
    employee={employee}  
  />  
))}
```

Using the array index as a key can cause problems when items are inserted, deleted, or reordered.

For example, an input's state might remain associated with the wrong employee after sorting.

Correct example

JavaScript

```
{employees.map(employee => (  
  <EmployeeRow  
    key={employee.id}  
    employee={employee}  
  />  
))}
```

```
    />  
  ,   ) ) }
```

Keys are scoped to siblings and do not need to be globally unique.

Advanced example: Resetting a form

JavaScript

```
<CustomerForm  
  key={customerId}  
  customerId={customerId}  
/>
```

Changing `customerId` changes the key, intentionally remounting the form and resetting its local state.

Interview-ready answer: Reconciliation compares element types and keys to determine identity. Stable keys prevent incorrect state preservation in dynamic lists, while intentionally changing a key can reset component state.

Part 2: Advanced React and Real-World Scenarios

11. How do you implement lazy loading and code splitting?

Application performance

Answer

Code splitting divides a JavaScript application into smaller bundles that can be downloaded when required instead of loading everything initially.

React provides `lazy` for loading component code dynamically and `Suspense` for displaying a fallback while it loads.

Practical example: Enterprise dashboard

JavaScript

```
import { lazy, Suspense } from "react";  
  
const Dashboard = lazy(() => import("./Dashboard"));  
const Reports = lazy(() => import("./Reports"));  
  
function App() {
```

```

return (
  <Suspense fallback={<div>Loading module...</div>}>
    <Dashboard />
    <Reports />
  </Suspense>
);
}

```

In this example, both components are rendered immediately, so both modules are requested. To gain route-level benefits, load a component only when its route is active.

Example: Lazy loading with React Router

JavaScript

```

import { lazy, Suspense } from "react";
import {
  BrowserRouter,
  Routes,
  Route
} from "react-router-dom";

const Dashboard = lazy(() => import("./Dashboard"));
const Reports = lazy(() => import("./Reports"));

function App() {
  return (
    <BrowserRouter>
      <Suspense fallback={<p>Loading page...</p>}>
        <Routes>
          <Route path="/" element={<Dashboard />} />
          <Route path="/reports" element={<Reports />} />
        </Routes>
      </Suspense>
    </BrowserRouter>
  );
}

```

Now the Reports module is not requested through this route until the user navigates to `/reports`, unless it has been prefetched or loaded elsewhere.

Production considerations: Use meaningful loading indicators, error boundaries for failed chunk loading, and route prefetching where appropriate.

Interview-ready answer: I implement route-level code splitting with dynamic imports, `React.lazy`, and `Suspense`. This reduces the initial JavaScript bundle by loading feature modules when needed.

12. How do you handle API calls and caching in React?

Real-world API integration

Answer

A production API integration must handle loading, errors, caching, stale data, concurrent requests, and cache invalidation.

For simple applications, `fetch` with `useEffect` may be sufficient. For complex applications, TanStack Query or a framework's data-loading APIs can simplify these responsibilities.

Practical example: Customer API caching

JavaScript

```
import {
  useQuery,
  useQueryClient
} from "@tanstack/react-query";

function CustomerList() {
  const queryClient = useQueryClient();

  const {
    data: customers = [],
    isPending,
    isFetching,
    error
  } = useQuery({
    queryKey: ["customers"],
    queryFn: async () => {
      const response = await fetch("/api/customers");

      if (!response.ok) {
        throw new Error("API request failed");
      }

      return response.json();
    },
    staleTime: 60_000
  });

  if (isPending) return <p>Loading...</p>;
  if (error) return <p>Unable to load customers</p>;

  return (
    <>
      <button
        disabled={isFetching}
        onClick={() =>
```

```

        queryClient.invalidateQueries({
          queryKey: ["customers"]
        })
      }
    >
    Refresh
  </button>

  { customers.map(customer => (
    <p key={customer.id}>{customer.name}</p>
  )) }
</>
);
}

```

The example assumes a `QueryClientProvider` is configured at the application root.

How caching works

- 1 First request fetches customers from the API.
- 2 Results are stored under the query key `["customers"]`.
- 3 Data remains fresh for 60 seconds under this configuration.
- 4 Invalidation marks the query stale and triggers refetching for active matching queries.

Senior-level point: A successful mutation should invalidate or update the affected queries. Cache keys must also distinguish parameters such as customer ID, search filters, and pagination.

Interview-ready answer: I use a server-state solution to manage caching, request deduplication, stale data, and background refetching. After mutations, I invalidate or update the relevant cached queries to maintain consistency.

13. How do you prevent race conditions in React API calls?

Advanced scenario

Answer

A race condition occurs when asynchronous requests complete in a different order than they were started.

Consider a customer search:

Example: Out-of-order API responses

Request A

Search: "John"

Slow

Request B completes first, then Request A completes and incorrectly overwrites the newer results.

Solution: AbortController with useEffect

JavaScript

```
import { useState, useEffect } from "react";

function CustomerSearch() {
  const [search, setSearch] = useState("");
  const [customers, setCustomers] = useState([]);

  useEffect(() => {
    const controller = new AbortController();

    async function loadCustomers() {
      try {
        const response = await fetch(
          `/api/customers?search=${encodeURIComponent(search)}`,
          { signal: controller.signal }
        );

        if (!response.ok) {
          throw new Error("Search failed");
        }

        const data = await response.json();

        if (!controller.signal.aborted) {
          setCustomers(data);
        }
      } catch (error) {
        if (error.name !== "AbortError") {
          console.error(error);
        }
      }
    }

    loadCustomers();

    return () => controller.abort();
  }, [search]);

  return (
    <>
```

```

<input
  value={search}
  onChange={e => setSearch(e.target.value)}
/>

{customers.map(customer => (
  <p key={customer.id}>{customer.name}</p>
))}
</>
);
}

```

When the search term changes, React runs the cleanup for the previous effect, aborting its request.

Additional production considerations: Debounce high-frequency searches, handle errors explicitly, and ensure older responses cannot overwrite newer data even when cancellation is unsupported.

Interview-ready answer: I prevent stale API responses by cancelling obsolete requests with `AbortController` or ignoring results from outdated requests. For server-state management, I use a query library that supports cancellation and request coordination.

14. What are custom hooks, and when should you create them?

Code reusability

Answer

A custom hook is a JavaScript function whose name begins with `use` and which can call other React hooks.

Custom hooks extract reusable stateful behavior without duplicating logic across components.

Practical example: `useDebounce`

JavaScript

```

import { useEffect, useState } from "react";

function useDebounce(value, delay = 500) {
  const [debouncedValue, setDebouncedValue] =
    useState(value);

  useEffect(() => {

```

```

    const timer = setTimeout(() => {
      setDebounceValue(value);
    }, delay);

    return () => clearTimeout(timer);
  }, [value, delay]);

  return debounceValue;
}

export default useDebounce;

```

Using the hook in a search component

JavaScript

```

import { useState, useEffect } from "react";
import useDebounce from "../useDebounce";

function Search() {
  const [search, setSearch] = useState("");

  const debounceSearch = useDebounce(search, 500);

  useEffect(() => {
    if (!debounceSearch.trim()) return;

    console.log("Search API:", debounceSearch);
  }, [debounceSearch]);

  return (
    <input
      value={search}
      onChange={e => setSearch(e.target.value)}
      placeholder="Search customers"
    />
  );
}

```

Instead of triggering a search for every keystroke, the effect runs when the debounced value changes.

In a real implementation, replace the logging with an API request and handle cancellation.

Important: Custom hooks share reusable logic, but each call has its own state unless the hook explicitly connects to shared state.

Interview-ready answer: I create custom hooks to encapsulate reusable stateful logic such as debouncing, subscriptions, API integration, or browser events. This improves

15. What is the difference between controlled and uncontrolled components?

Forms

Answer

In controlled components, React state controls the current value of a form element.

In uncontrolled components, the DOM maintains the input's current value, which can be accessed through a ref or form submission.

Example 1: Controlled component

JavaScript

```
import { useState } from "react";

function LoginForm() {
  const [email, setEmail] = useState("");

  return (
    <form onSubmit={e => {
      e.preventDefault();
      console.log(email);
    }}>
      <input
        type="email"
        value={email}
        onChange={e => setEmail(e.target.value)}
      />

      <button type="submit">Login</button>
    </form>
  );
}
```

React state is the source of truth for the email field.

Example 2: Uncontrolled component

JavaScript

```
import { useRef } from "react";

function LoginForm() {
  const emailRef = useRef(null);
```

```
function handleSubmit(e) {
  e.preventDefault();

  console.log(emailRef.current.value);
}

return (
  <form onSubmit={handleSubmit}>
    <input
      type="email"
      ref={emailRef}
      defaultValue=""
    />

    <button type="submit">Login</button>
  </form>
);
}
```

Here, the browser maintains the input value.

Controlled

Uses `value` and `onChange`

React manages current value

Convenient for live validation

Uncontrolled

Uses `defaultValue` or `refs`

DOM manages current value

Useful for simple forms and integrations

Senior-level point: File inputs are generally uncontrolled because their selected file values cannot be programmatically set like ordinary text inputs.

Interview-ready answer: Controlled components use React state as the source of truth, enabling validation and conditional UI updates. Uncontrolled components retain values in the DOM and are useful when direct state synchronization is unnecessary.

16. How do you implement authentication and authorization in React?

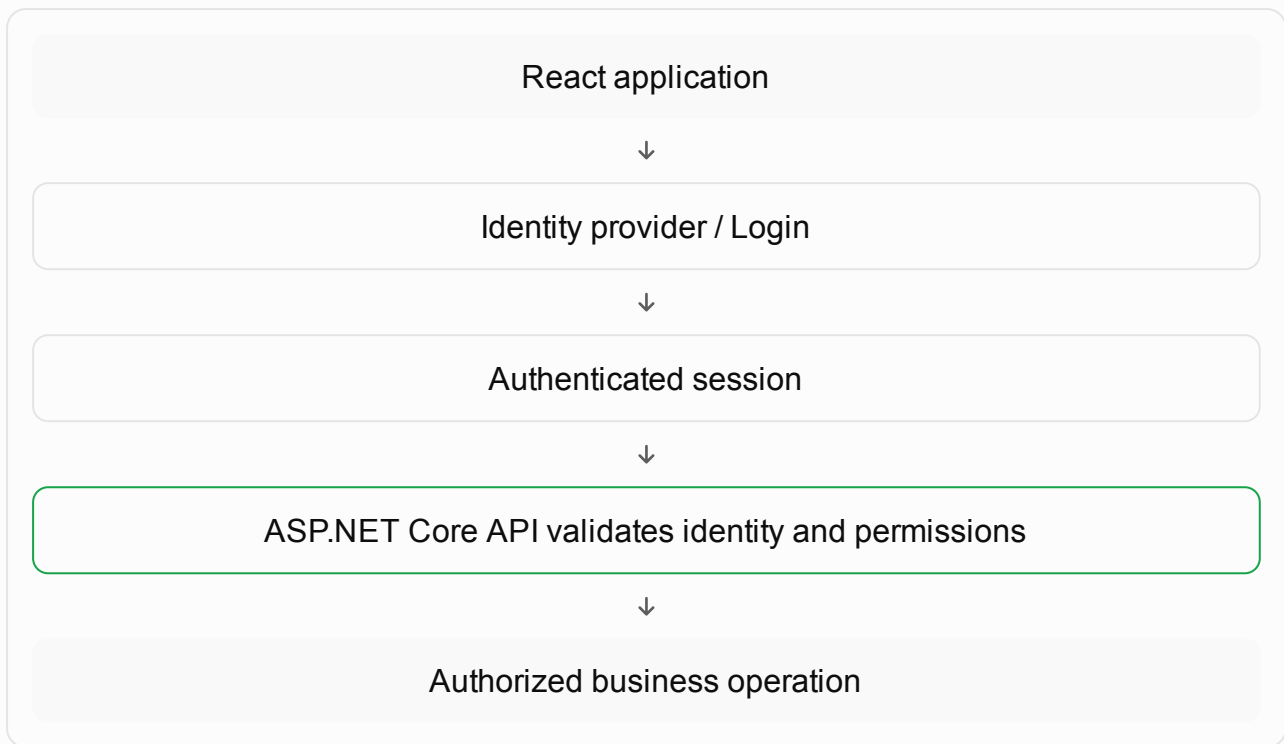
Security and architecture

Answer

Authentication establishes who a user is. Authorization determines which operations that user is permitted to perform.

React can manage the user interface and authentication flow, but the backend must enforce authorization.

Typical enterprise architecture



Example: Protected route

This example assumes `useAuth()` is an application-specific hook that returns an authenticated user and loading status.

JavaScript

```
import { Navigate, Outlet } from "react-router-dom";
import { useAuth } from "../AuthContext";

function ProtectedRoute() {
  const { user, loading } = useAuth();

  if (loading) {
    return <p>Checking session...</p>;
  }

  if (!user) {
    return <Navigate to="/login" replace />;
  }

  return <Outlet />;
}
```

Role-based UI example

JavaScript

```
function AdminPanel({ user }) {
  if (!user?.roles?.includes("Admin")) {
    return <p>Access denied</p>;
  }

  return <h2>Admin Dashboard</h2>;
}
```

secure the underlying API.

Backend enforcement using ASP.NET Core

C#

```
[Authorize(Roles = "Admin")]
[HttpDelete("customers/{id}")]
public IActionResult DeleteCustomer(int id)
{
    // Perform authorized deletion.
    return NoContent();
}
```

The backend must configure authentication and authorization middleware and validate the user's identity and roles.

Security considerations: Use an established OpenID Connect/OAuth implementation. For browser applications, consider a backend-for-frontend pattern with secure HttpOnly session cookies, appropriate SameSite settings, and CSRF protection. Never rely on hidden UI controls for security.

Interview-ready answer: React handles login navigation, session-aware UI, and protected routes. I use a standard identity provider and enforce all authorization decisions in the backend. Client-side role checks are for user experience, not security enforcement.

17. How do you handle errors using Error Boundaries?

Reliability

Answer

Error Boundaries are React components that catch errors thrown while rendering their descendant component tree.

They can display fallback UI instead of allowing an entire application interface to disappear.

Practical example

JavaScript

```
import React from "react";

class ErrorBoundary extends React.Component {
  constructor(props) {
    super(props);

    this.state = { hasError: false };
  }

  static getDerivedStateFromError() {
    return { hasError: true };
  }

  componentDidCatch(error, info) {
    console.error("React error:", error, info);
  }

  render() {
```

Using the Error Boundary

JavaScript

```
<ErrorBoundary>
  <CustomerDashboard />
</ErrorBoundary>
```

If `CustomerDashboard` throws during rendering, the boundary displays its fallback UI.

What Error Boundaries do not catch

- Errors in event handlers.
- Errors in ordinary asynchronous callbacks.
- Errors during server-side rendering.
- Errors thrown inside the Error Boundary itself.

Handle API failures with explicit error state or your data-fetching library. Handle event-handler failures with normal JavaScript error handling.

Production consideration: Place boundaries around independent application sections so that one failed dashboard widget does not necessarily take down the entire page.

Interview-ready answer: Error Boundaries isolate rendering failures and display fallback interfaces. I place them around major features, log errors to monitoring systems, and handle asynchronous and event-handler errors separately.

18. How do you test React components?

Testing and quality

Answer

React testing should focus on observable behavior rather than internal implementation details.

A common stack includes React Testing Library, Jest or Vitest, and Mock Service Worker for API mocking.

Component to test

JavaScript

```
import { useState } from "react";

export function Counter() {
  const [count, setCount] = useState(0);

  return (
    <>
      <p>Count: {count}</p>

      <button onClick={ () => setCount(c => c + 1)}>
        Increment
      </button>
    </>
  );
}
```

Unit test using React Testing Library and Jest

JavaScript

```
import { render, screen, fireEvent } from
  "@testing-library/react";

import "@testing-library/jest-dom";
import { Counter } from "../Counter";

test("increments count when clicked", () => {
  render(<Counter />);

  const button = screen.getByRole("button", {
    name: "Increment"
  });

  fireEvent.click(button);
});
```

```
expect(
  screen.getByText("Count: 1")
).toBeInTheDocument();
});
```

The test renders the component, finds the button by its accessible role, clicks it, and verifies the visible result.

Example: Testing an asynchronous API result

JavaScript

```
test("displays customers", async () => {
  render(<CustomerList />);

  expect(
    await screen.findByText("John Smith")
  ).toBeInTheDocument();
});
```

This second test assumes `CustomerList` is rendered with any required providers and that the API is mocked to return John Smith.

Testing strategy for enterprise applications: Test individual components, feature workflows, API integrations, authentication boundaries, and critical end-to-end user journeys.

Interview-ready answer: I test components through user-visible behavior using React Testing Library, mock external dependencies, and cover critical workflows with integration and end-to-end tests. I avoid tests that depend unnecessarily on component implementation details.

19. How would you design a scalable React application architecture?

Senior architect scenario

Answer

A scalable React application should organize features clearly, separate responsibilities, provide consistent API access, and support independent testing.

Consider a healthcare application containing registration, appointments, billing, and reporting modules.

Suggested feature-based folder structure

```
src/
├── app/
│   ├── App.jsx
│   ├── router.jsx
│   └── providers.jsx
├── features/
│   ├── patients/
│   │   ├── components/
│   │   ├── hooks/
│   │   ├── api/
│   │   └── pages/
│   ├── appointments/
│   ├── billing/
│   └── reports/
├── shared/
│   ├── components/
│   ├── hooks/
│   └── utils/
└── services/
    └── httpClient.js
```

This structure groups related code by business capability rather than placing every component, hook, and API function into large global directories.

Example: Centralized HTTP client

JavaScript

```
export async function apiGet(url) {
  const response = await fetch(url, {
    credentials: "include"
  });

  if (!response.ok) {
    throw new Error(`HTTP error: ${response.status}`);
  }

  return response.json();
}
```

This example assumes the backend uses cookie-based authentication and has the necessary CSRF and CORS protections configured.

Feature-specific API layer

JavaScript

```
import { apiGet } from "../../services/httpClient";

export function getPatients() {
  return apiGet("/api/patients");
}

export function getPatient(id) {
  return apiGet(
    `/api/patients/${encodeURIComponent(id)}`
  );
}
```

Architecture decisions to explain in an interview

- Organize modules by business domain.
- Keep presentation separate from data-access concerns.
- Centralize cross-cutting infrastructure without creating a single oversized utility layer.
- Use clear state-management boundaries.
- Implement authentication and backend authorization.
- Add automated tests, monitoring, and performance measurement.

Micro-frontends may be appropriate when independently deployed teams and applications justify their operational complexity; they are not automatically necessary for large React applications.

Interview-ready answer: I design React applications around business features, isolate API integration and shared infrastructure, define clear state ownership, and enforce consistent testing and security practices. I choose architectural complexity according to team and deployment requirements.

20. Explain React Server Components versus Client Components.

Advanced React architecture

Answer

React Server Components allow supported frameworks to render components in a server environment and send their output to the client.

Client Components provide interactivity and can use browser-specific APIs and client-side hooks such as `useState`.

React Server Components require framework or bundler integration. The following examples use the Next.js App Router.

Example 1: Server Component

JavaScript

```
// app/customers/page.jsx
// Server Component by default in Next.js App Router

export default async function CustomersPage() {
  const response = await fetch(
    "https://api.example.com/customers"
  );

  if (!response.ok) {
    throw new Error("Unable to load customers");
  }

  const customers = await response.json();

  return (
    <div>
      <h1>Customers</h1>

      {customers.map(customer => (
        <p key={customer.id}>{customer.name}</p>
      ))}
    </div>
  );
}
```

This component can fetch data in the server environment without shipping its component implementation as client-side JavaScript.

Example 2: Client Component

JavaScript

```
// app/customers/Counter.jsx

"use client";

import { useState } from "react";

export default function CustomerCounter() {
  const [count, setCount] = useState(0);

  return (
    <button onClick={() => setCount(c => c + 1)}>
```

```
        Selected: {count}
      </button>
    );
  }
}
```

The `"use client"` directive establishes a client boundary in frameworks such as Next.js.

Combining both approaches

JavaScript

```
// app/customers/page.jsx

import CustomerCounter from "../CustomerCounter";

export default function CustomersPage() {
  return (
    <>
      <h1>Customer Dashboard</h1>
      <CustomerCounter />
    </>
  );
}
```

A Server Component can render a Client Component. The framework handles the client boundary and hydration.

Server Components	Client Components
Execute in a server environment	Support browser interactivity
Can access server-side resources	Can use state and effect hooks
Component code is not shipped as client JS	Client bundle includes necessary code
Cannot attach client event handlers directly	Support event handlers

Important distinction: Server Components are not the same as traditional server-side rendering. Client Components can also be pre-rendered to HTML on the server and subsequently hydrated in the browser.

Interview-ready answer: Server Components enable server-side data access and reduce the amount of component JavaScript sent to the browser. Client Components handle interactive behavior. I use server boundaries for suitable data-driven UI and client boundaries where state, effects, or browser interaction are required.

Final preparation: Test your understanding

5 questions

0/5 answered

1. Which hook is appropriate for managing complex state transitions?

- ☐ useEffect
- ☐ useReducer
- ☐ useMemo
- ☐ useRef

2. What does React.memo primarily do?

- ☐ Cache API responses
- ☐ Prevent all component renders
- ☐ Allow skipping renders when props are unchanged
- ☐ Cache function references

3. How should you prevent stale API responses from overwriting newer results?

- ☐ Use useMemo
- ☐ Use AbortController or ignore outdated results
- ☐ Use array indexes as keys
- ☐ Disable Strict Mode

4. Where must authorization be enforced?

- ☐ Only in React routes
- ☐ Only by hiding buttons
- ☐ On the backend
- ☐ Only in Redux

5. Which tool is designed to manage cached server state?

- ☐ useRef

- ☐ React.memo
- ☐ TanStack Query
- ☐ useCallback

Check my answers

Source note: The uploaded interview material includes Angular and ASP.NET Core topics but does not substantiate the React-specific questions or company frequencies. The React explanations and examples above are additional technical study material, not claims about verified company interview questions.